13장

라이브러리 -> 잘 사용

랭그래프를 활용한 멀티에이전트 RAG 만들기

다중(여러 개) Node VectorDB

12장에서 랭그래프로 기본 챗봇을 만들면서 '오픈AI API나 랭체인을 활용해도 충분한데 굳이 이렇게 복잡하게 구현할 필요가 있을까?' 라고 의문이 생겼을 수도 있습니다. 하지만 해결해야 할 문제가 많고 복잡해지면 랭그래프의 장점이 빛을 발합니다. 이 장에서는 랭그래프를 활용해 RAG를 만들 때 더할 수 있는 부가 기능을 살펴보겠습니다.

13-1 랭그래프 기반 RAG를 위한 사전 작업 VectorDB 필요
13-2 라우터 알아보기 Conditional_Edge
13-3 랭그래프로 RAG 에이전트 만들기 Node 생성
13-4 그래프 정의하기 분리된 기능

13-1 랭그래프 기반 RAG를 위한 사전 작업

기능

작은 단위의 인공지능 프로그램들이 서로 협업하여 작업을 수행하는 멀티에이전트에 대해 알아보겠습니다. 그리고 멀티에이전트와 랭그래프를 활용해 RAG의 한계를 개선해보겠습니다.

Node
+LLM

멀티에이전트 시스템과 정확한 가이드

랭체인, 랭그래프와 관련된 기술은 빠르게 업데이트되고 있고 공식 문서, 블로그, 논문 등의 기술 문서가 계속해서 공개되고 있습니다. 하지만 현실의 복잡한 작업을 인공지능이 효과적으로 수행하도록 만드는 것은 여전히 쉽지 않습니다. 많은 사람이 챗GPT에게 보고서 작성 같은 복잡한 작업을 요청했다가 기대에 못 미치는 결과에 실망하곤 합니다. 이럴 때 '아직 인공지능은 멀었구나.'라고 생각할 수 있지만 원하는 결과를 얻기 위한 가이드가 부족했기 때문일 수도 있습니다.

복잡한 작업을 인공지능에 맡길 때 정확한 가이드가 부족하면 원하는 결과를 얻기 어렵습니다. 이런 한계는 사람에게 일을 맡길 때도 마찬가지입니다. 예를 들어 맥도날드의 '빅맥'은 세계 어디서나 맛과 모양이 비슷하고 대부분의 사람이 빅맥의 맛과 모양을 알고 있습니다. 하지만 제가 맥도날드 매장의 주방에 들어가서 당장 빅맥을 만든다면 흉내는 내더라도 그 맛과 모양을 장담할 수 없을 겁니다. 이때 '왜 이렇게 못 만들었냐'고 저의 지능을 탓한다면 조금 억울할 것 같습니다. 아무도 저에게 빅맥을 어떻게 만들어야 하는지 정확하게 알려주지 않았으니까요.

작업자에게 정확한 가이드를 제공하기 위해서는 업무를 세분화하고 구체적으로 정의해야 합니다. 만약 업무를 수행하는 사람마다 다르게 해석할 여지가 있다면 가이드가 제대로 작성되었다고 할 수 없습니다. 예를 들어 '빅맥은 구운 빵 사이에 고기, 채소, 소스를 끼워 만든다.'라는 설명은 추상적입니다. '구운 빵'을 준비하는 과정도 빵을 보관하는 방법, 빵을 굽는 온도 등 여러 단계로 나누어 설명할 수 있습니다. 이처럼 복잡한 작업을 수행하기 위해서는 구체적으로 정의된 가이드가 필요합니다.

이런 복잡한 작업을 수행할 때 한 사람에게 모두 맡기기보다는, 업무를 세분화하여 전문적으로 수행할 수 있는 사람에게 맡기는 방식이 훨씬 효과적인 경우가 많습니다. 예를 들어 배달을 담당하는 사람은 배달만 잘하도록 하고 햄버거를 만드는 사람은 그 과정을 정확히 수행할 수 있도록 각각의 업무에 대해 명확한 가이드를 제공하는 것이죠. 만약 여러 작업 중 기대에 못 미치는 부분이 있다면 해당 작업의 가이드를 별도로 개선하면 됩니다. 결과적으로 각 업무에 최적화된 사람이 효율적으로 작업을 수행할 수 있습니다.

인공지능에 복잡한 작업을 요청할 때도 마찬가지입니다. 하나의 AI 에이전트에게 모든 작업을 맡기는 것보다 각각의 AI 에이전트에게 단순한 업무 단위로 명확하게 지시를 내리고 이를 결합한 워크플로workflow를 만들면 마치 각 분야의 전문가들이 협업하는 것처럼 시너지를 낼 수 있습니다. 이런 방식으로 AI 에이전트에게 작업을 분배하고 서로 협력하여 복잡한 작업을 처리하는 시스템을 멀티에이전트multi-agent라고 합니다.

✦ 워크플로는 특정 목표를 달성하기 위해 여러 작업이나 프로세스를 수행하는 작업 흐름을 말합니다.

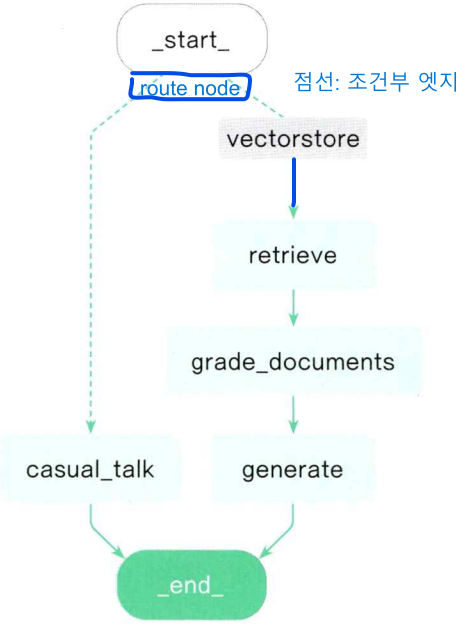
RAG의 한계 개선하기

09장에서 만들었던 RAG의 흐름을 떠올려 봅시다. 사용자가 질문하면 예외 없이 리트리버를 활용해 관련 문서들을 가져오고, 그 문서들을 바탕으로 답변합니다. 사용자가 '그렇구나. 잘 찾아왔네. 고마워.'라고 감사를 표하는 말을 하거나 '잘 지냈어?'와 같은 일상적인 질문을 하는 경우에도 앞에서 만든 RAG는 관련 문서를 찾아와서 답변하므로 시간과 토큰 비용을 낭비하죠. 이런 문제를 해결하는 방법을 배워 보겠습니다.



랭그래프를 이용하면 사용자의 질문을 분석해서 RAG를 사용할지 말지 판단하고 RAG가 필요한 경우에만 검색을 사용할 수 있습니다. RAG를 실행할 때에도 필요하지 않은 청크들은 제외하고 답변을 생성하는 등 다양한 작업을 추가할 수 있습니다.

오른쪽 그림은 이번 절에서 우리가 만들 간단한 랭그래프 프로그램의 구조입니다. 사용자의 질문에 답변하는 데 RAG가 필요한지 판단하고, 필요하지 않다면 RAG를 수행하지 않는 causal_talk 노드로 가서 답변을 생성합니다. RAG가 필요하다면 리트리버로 질문과 관련된 청크를 가져오고, 이 청크가 사용자의 질문에 답변하는 데 정말 쓸모가 있는지 판단합니다. 그리고 쓸모 있는 청크만 활용해 언어 모델로 답변을 생성합니다.



RAG가 필요한지 판단하고 동작하는 랭그래프의 구조

Do it! 실습 PDF 전처리하고 벡터 DB 만들기

결과 파일: chap13/sec01/rag_with_langgraph.ipynb

이제 직접 프로그램을 만들며 랭그래프에 기반한 RAG를 구현해 봅시다.

✦ 이 절의 내용은 랭그래프 공식 문서의 예제를 이 책의 내용에 맞게 일부는 간략화하고 일부는 확장하여 구성했습니다(Adaptive RAG: https://langchain-ai.github.io/langgraph/tutorials/rag/langgraph_adaptive_rag/#compile-graph).

1. 주피터 노트북 파일 `rag_with_langgraph.ipynb`을 만들고 RAG에 필요한 요소들을 하나씩 채워 가겠습니다. 09장에서 사용한 PDF 파일들을 사용하겠습니다. 09장에서 사용한 data 폴더를 프로젝트 폴더에 붙여 넣고 다음처럼 입력해 PDF 파일들이 잘 있는지 확인해 봅시다.

사용할 PDF 파일 확인하기 — rag_with_langgraph.ipynb (1)

```
from glob import glob

for g in glob('../data/*.pdf'):
    print(g)
```

다음과 같이 PDF 경로가 출력되어야 문제없이 진행됩니다. 물론 여러분이 원하는 PDF 파일을 사용해도 됩니다.

```
../data\2040_seoul_plan.pdf
../data\OneNYC_2050_Strategic_Plan.pdf
```

2. PDF 파일을 텍스트로 읽고 청크 단위로 텍스트를 잘라 리스트로 담아 반환하는 `read_pdf_and_split_text` 함수를 만들겠습니다. 이 함수의 코드는 09-2절 rag_practice.ipynb 파일에서 가져와 변수명만 수정했습니다.

read_pdf_and_split_text 함수 만들기 — rag_with_langgraph.ipynb (2)

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

def read_pdf_and_split_text(pdf_path, chunk_size=1000, chunk_overlap=100):
    """
    주어진 PDF 파일을 읽고 텍스트를 분할합니다.
```

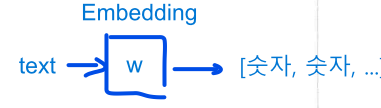
```
    매개변수:
        pdf_path (str): PDF 파일의 경로.
        chunk_size (int, 선택적): 각 텍스트 청크의 크기. 기본값은 1000입니다.
        chunk_overlap (int, 선택적): 청크 간의 중첩 크기. 기본값은 100입니다.
    반환값:
        list: 분할된 텍스트 청크의 리스트.
    """
    print(f"PDF: {pdf_path} ----------------------------")

    pdf_loader = PyPDFLoader(pdf_path)
    data_from_pdf = pdf_loader.load()

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=chunk_overlap
    )

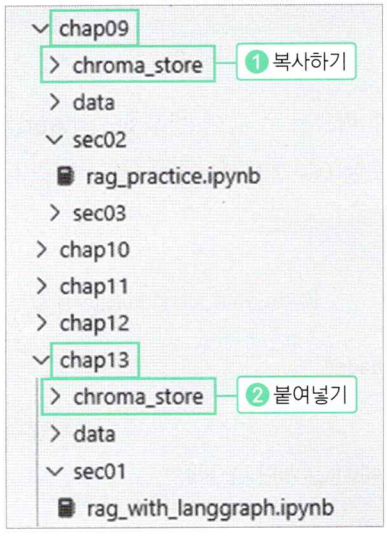
    splits = text_splitter.split_documents(data_from_pdf)

    print(f"Number of splits: {len(splits)}\n")
    return splits
```



text ----------> vector 저장

3. 앞서 만든 read_pdf_and_split_text 함수를 이용해 잘라 놓은 청크들을 임베딩해서 크로마 DB에 저장하겠습니다. 임베딩하는 비용과 시간을 절약하고 싶다면 09장에서 만든 chroma_store 폴더를 현재 실습 폴더로 복사하면 됩니다. 이미 크로마 DB로 만든 벡터 DB가 chroma_store 폴더에 저장되어 있는 경우에는 그 DB를 읽어 vectorstore에 담습니다.



그렇지 않은 경우에는 PDF를 하나씩 열어 청크 단위로 쪼갠 뒤 이를 이용해 새로운 벡터 DB를 생성합니다. 이때 너무 많은 양의 청크를 한 번에 임베딩하려고 하면 오픈AI API에서 '너무 많은 양의 문서를 한 번에 요청한다'는 내용의 오류 메시지가 출력되며 제대로 작동되지 않습니다.

```
BadRequestError: Error code: 400 - {'error': {'message': 'Requested 637068 tokens, max
600000 tokens per request', 'type': 'max_tokens_per_request', 'param': None, 'code':
'max_tokens_per_request'}}
```

4. 청크를 100개씩 나눠서 처리하겠습니다. 우선 vectorstore를 None으로 설정합니다. 처음 실행될 때는 아직 벡터 DB가 없는 상태이므로 Chroma.from_documents로 새로 설정하고 그다음부터는 이미 생성된 vectorstore에 추가합니다.

vectorstore 설정하고 청크 처리하기 — rag_with_langgraph.ipynb (3)

```
from langchain_openai import OpenAIEmbeddings   -> API_KEY
from langchain_chroma import Chroma
import os

##### vectorstore 설정 #####                    [3072]
embedding = OpenAIEmbeddings(model='text-embedding-3-large')

persist_directory='../chroma_store'

if os.path.exists(persist_directory):
    print("Loading existing Chroma store")
    vectorstore = Chroma(  -> DB 연결 -> 명령
        persist_directory=persist_directory,
        embedding_function=embedding
    )          pdf -> split -> vector
else:  생성 -> data -> wjwkd
    print("Creating new Chroma store")

    vectorstore = None
    for g in glob('../data/*.pdf'):
        chunks = read_pdf_and_split_text(g)
        # 100개씩 나눠서 저장
        for i in range(0, len(chunks), 100):
            if vectorstore is None:
                vectorstore = Chroma.from_documents(
                    documents=chunks[i:i+100],
```

pdf_path
chunk_size=100, chunk_overlap=100

```
                embedding=embedding,
                persist_directory=persist_directory
            )
        else:
            vectorstore.add_documents(
                documents=chunks[i:i+100]
            )
```

5. 이제 벡터 DB가 잘 작동하는지 확인해 봅시다. 이 실습에서 사용한 PDF 파일은 서울, 뉴욕의 도시 계획에 관한 문서이므로 '서울 온실가스 저감 계획'이라는 쿼리를 사용해 관련 문서를 가져오도록 테스트했습니다.

관련 문서 검색하기 — rag_with_langgraph.ipynb (4)

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

chunks = retriever.invoke("서울 온실가스 저감 계획")

for chunk in chunks:
    print(chunk.metadata)
    print(chunk.page_content)
```

실행한 결과 다음과 같이 관련 내용을 잘 검색해 옵니다.

```
{'author': 'SI', 'creationdate': '2024-12-12T18:16:11+09:00', 'creator': 'Hwp 2020
11.0.0.5178', 'moddate': '2024-12-12T18:16:11+09:00', 'page': 63, 'page_label': '64',
'pdfversion': '1.4', 'producer': 'Hancom PDF 1.3.0.542', 'source': '../data\\2040_
seoul_plan.pdf', 'total_pages': 205}
56제2장 미래상과 목표
6. 미래위기를 준비하는, '탄소중립 안전도시 구축'1) 배경전(全) 지구적인 기후변화에 대응하기 위한 대
도시 차원의 대응 필요Ÿ서울시 2017년 온실가스 배출량은 46,685천 톤CO2eq로 2005년 배출량에 비해
5.6%(276만 톤CO2eq) 감소하였으며, 서울의 1인당 온실가스 배출량은 4.7CO2eq로 국가의 13.8톤CO2eq
에 비해 낮은 수준이다. 또한 2014년 기준 서울의 1인당 온실가스 배출량은 4.5CO2eq로 세계 주요 도시에
비해 낮은 수준을 보인다.7)
(... 생략 ...)
{'author': 'SI', 'creationdate': '2024-12-12T18:16:11+09:00', 'creator': 'Hwp 2020
11.0.0.5178', 'moddate': '2024-12-12T18:16:11+09:00', 'page': 64, 'page_label': '65',
'pdfversion': '1.4', 'producer': 'Hancom PDF 1.3.0.542', 'source': '../data\\2040_
seoul_plan.pdf', 'total_pages': 205}
```

```
수립시 주요하게 고려한다.장기적·구조적 전환을 위해 도시계획 단계에서의 탄소중립·기후위기 적응대책 반영 탄소중립 및 기후위기 대책을 개별 사업 단위로 마련해 왔으나, 앞으로는 도시계획 및 도시 인프라 전반에서 더욱 장기적이고 구조적인 접근이 필요하다. 토지이용 ... 생략 ...
```

6. 챗봇으로 활용하기 위해 다음과 같이 언어 모델을 설정합니다.

언어 모델 설정하기 — rag_with_langgraph.ipynb (5)

```
from langchain_openai import ChatOpenAI

# 모델 초기화
model = ChatOpenAI(model="gpt-4o-mini")
model.invoke('안녕하세요!')
```

이 셀을 실행하면 다음과 같이 AIMessage가 반환됩니다.

```
AIMessage(content='안녕하세요! 어떻게 도와드릴까요?', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 11, 'prompt_tokens': 10, 'total_tokens': 21, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint': 'fp_06737a9306', 'finish_reason': 'stop', 'logprobs': None}, id='run-b1726e7a-35e9-4a5b-af5c-d82010778adb-0', usage_metadata={'input_tokens': 10, 'output_tokens': 11, 'total_tokens': 21, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}})
```

13-2 라우터 알아보기

사용자의 질문에 따라 RAG 사용 여부를 판단하는 기능인 라우터를 알아보고, 이를 13-1절에서 만든 챗봇에 설정해 보겠습니다.

라우터

지금 만드는 챗봇은 사용자의 질문에 따라 RAG의 사용 여부를 판단해야 합니다. 예를 들어 사용자가 '잘 지냈어?'라고 입력하면 일상적인 대화를 하고, '서울시의 온실가스 저감 계획은 뭐야?'와 같은 질문을 하면 관련 문서를 활용해 RAG를 실행해야 합니다. 이렇게 하지 않으면 사용자가 단순한 인사말을 할 때도 불필요하게 문서를 검색해서 토큰을 낭비하고 그 내용을 대화 내용에 포함시켜 불필요한 말을 답변에 추가할 수 있으니까요.

이런 판단을 처리하려면 라우터router 기능이 필요합니다. 라우터는 입력한 내용에 따라 여러 개의 실행 경로 중에서 적절한 경로를 결정해 다음 노드를 선택하는 기능을 합니다. 일반적으로 언어 모델의 응답이나 특정 조건에 따라 다르게 동작해야 할 때 라우터를 사용합니다.

Do it! 실습 챗봇에 라우터 설정하기

결과 파일: sec01/rag_with_langgraph.ipynb

GPT와 같은 언어 모델이 사용자의 입력을 일상적인 대화인지 RAG가 필요한 질문인지 판단하는 기능을 구현해 보겠습니다. 일상적인 대화라면 casual_talk를 반환하고 RAG가 필요하면 vector_store를 반환합니다. 그리고 이 판단에 따라 conditional_edge를 이용해 어떤 노드를 활용할지를 판단하게 됩니다. 바로 앞 실습에서 사용한 주피터 노트북 파일에 이어서 작성하겠습니다.

1. 랭체인을 처음 배울 때 다뤘던 `BaseModel`을 이용해 `RouteQuery`라는 클래스를 만듭니다. `datasource` 필드는 `Literal` 타입으로 설정하여 `vector_store`와 `casual_talk` 중 하나만 선택하도록 제한합니다. 단순히 GPT에게 프롬프트로 상황에 따라 판단하고 `vector_store` 혹은 `casual_talk` 중 하나를 답변하게 할 수도 있지만 그렇게 하면 '`vector_store`입니다.'와 같은 불필요한 내용을 답변에 포함시킬 수 있습니다.

라우터 설정하기 rag_with_langgraph.ipynb (6)

```
# 라우터 설정
from langchain_core.prompts import ChatPromptTemplate
from typing import Literal # 문자열 리터럴 타입을 지원하는 typing 모듈의 클래스
from pydantic import BaseModel, Field

# Data model
class RouteQuery(BaseModel):
    """사용자 쿼리를 가장 관련성이 높은 데이터 소스로 라우팅합니다."""

    datasource: Literal["vectorstore", "casual_talk"] = Field(
        ...,
        description="""
        사용자 질문에 따라 casual_talk 또는 vectorstore로 라우팅합니다.
        - casual_talk: 일상 대화를 위한 데이터 소스. 사용자가 일상적인 질문을 할 때 사용합니다.
        - vectorstore: 사용자 질문에 답하기 위해 RAG로 vectorstore 검색이 필요한 경우 사용합니다.
        """,
    )
```

2. RouterQuery는 model.with_structured_output(RouterQuery)로 정의하여 둘 중 하나만 출력되도록 합니다. router_system으로 시스템 프롬프트를 작성해 사용자의 질문을 판단하게 합니다. ChatPromptTemplate.from_messages를 이용해 route_prompt로 프롬프트 템플릿을 생성합니다. 이 프롬프트 안의 "{question}"은 question_router 체인을 통해 사용자의 질문을 받는 빈칸입니다.

사용자 질문 받고 판단하기 rag_with_langgraph.ipynb (7)

```
# 특정 모델을 structured output(구조화된 출력)과 함께 사용하기 위해 설정
structured_llm_router = model.with_structured_output(RouteQuery)

router_system = """
당신은 사용자의 질문을 vectorstore 또는 casual_talk으로 라우팅하는 전문가입니다.
- vectorstore에는 서울, 뉴욕의 발전계획과 관련된 문서가 포함되어 있습니다. 이 주제에 대한 질문에는
vectorstore를 사용하십시오.
- 사용자의 질문이 일상 대화에 관련된 경우 casual_talk을 사용하십시오.
"""

# 시스템 메시지와 사용자의 질문을 포함하는 프롬프트 템플릿 생성
route_prompt = ChatPromptTemplate.from_messages([
```

```
    ("system", router_system),
    ("human", "{question}"),
])

# 라우터 프롬프트와 구조화된 출력 모델을 결합한 객체
question_router = route_prompt | structured_llm_router
```

3. 마지막으로 이 코드가 잘 작성되었는지 확인하기 위해 '서울 온실가스 저감 계획은 무엇인가요?' 질문과 '잘 지냈어?'라는 질문을 각각 넣습니다.

결과 출력하기 rag_with_langgraph.ipynb (8)

```
print(
    question_router.invoke({
        "question": "서울 온실가스 저감 계획은 무엇인가요?"
    })
)

print(question_router.invoke({"question": "잘 지냈어?"}))
```

이 셀을 실행해 보니 똑똑하게 질문을 잘 구분해서 정해진 서식대로 판단 결과를 반환합니다.

```
datasource='vectorstore'
datasource='casual_talk'
```

13-3 랭그래프로 RAG 에이전트 만들기

여러 기능을 각각의 노드로 만들고 이 노드들을 연결하여 사용자의 요청에 대응할 수 있는 AI 에이전트로 발전시켜 봅시다.

Do it! 실습 관련 있는 청크만 필터링하기

결과 파일: sec01/rag_with_langgraph.ipynb

리트리버가 잘 작동하는지는 13-1절에서 이미 테스트했습니다. 리트리버는 벡터 DB에서 벡터 유사도를 기준으로 가장 관련 있는 문서 n개(현재는 5개)를 가져옵니다. 문서를 무조건 5개 가져오기 때문에 질문과 관련 없는 청크가 선택될 수 있습니다. 질문과 관련 없는 청크를 포함하는 경우 답변을 생성할 때 혼란을 일으킬 수 있으므로 미리 삭제하는 편이 좋습니다. 문서의 관련성을 판단하는 방법을 프롬프트로 작성하고, 판단 결과는 BaseModel과 .with_structured_output을 활용해 출력 형식을 제한해 보겠습니다. 앞에서 사용한 주피터 노트북 파일에 이어서 작성하세요.

Literal

1. BaseModel을 사용해 출력 방식을 yes 또는 no로만 제한하는 GradeDocuments 클래스를 만듭니다. GPT 모델을 그대로 쓰지 않고 .with_structured_output을 사용해 출력 형식을 제한한 structured_llm_grader를 만들어서 사용합니다.

GradeDocuments 클래스 만들기 — rag_with_langgraph.ipynb (9)

```
from langchain_core.prompts import PromptTemplate

class GradeDocuments(BaseModel):
    """검색된 문서가 질문과 관련성 있는지 yes 또는 no로 평가합니다."""

    binary_score: Literal["yes", "no"] = Field(
        description="문서가 질문과 관련이 있는지 여부를 'yes' 또는 'no'로 평가합니다."
    )

structured_llm_grader = model.with_structured_output(GradeDocuments)
```

이번에는 PromptTemplate을 이용해 프롬프트를 생성하겠습니다. 이전까지는 챗봇에서 많이 사용하는 ChatPromptTemplate을 주로 활용했는데 기존 대화 내용이 계속 이어질 필요가 없는 경우에는 PromptTemplate이 더 적합합니다.

2. `PromptTemplate.from_template`으로 프롬프트 메시지를 작성합니다. 리트리버가 가져온 청크가 질문과 관계 있으면 `yes` 아니면 `no`로 평가하도록 설정합니다. 그리고 `{document}`, `{question}`으로 빈칸을 만듭니다. 이 빈칸은 `for` 문 안에서 딕셔너리 형태로 채워집니다. 테스트를 위해 '서울시 자율주행 관련 계획'이라는 쿼리로 리트리버를 실행했습니다.

✦ 서울시의 도시 계획 문서와 관련이 적은 내용을 질문해야 관련 없는 청크까지 함께 가져옵니다.

프롬프트 작성하고 테스트하기 — rag_with_langgraph.ipynb (10)

```
grader_prompt = PromptTemplate.from_template("""
당신은 검색된 문서가 사용자 질문과 관련이 있는지 평가하는 평가자입니다. \n
문서에 사용자 질문과 관련된 키워드 또는 의미가 포함되어 있으면, 해당 문서를 관련성이 있다고 평가하십시오. \n
엄격한 테스트가 필요하지 않습니다. 목표는 잘못된 검색 결과를 걸러내는 것입니다. \n
문서가 질문과 관련이 있는지 여부를 나타내기 위해 'yes' 또는 'no'로 이진 점수를 부여하십시오.

Retrieved document: \n {document} \n\n
User question: {question}
""")

retrieval_grader = grader_prompt | structured_llm_grader
question = "서울시 자율주행 관련 계획"
documents = retriever.invoke(question)

for doc in documents:
    print(doc)
```

이 셀을 실행하면 다음과 같이 검색된 결과가 출력됩니다. 총 5개의 청크를 가져왔습니다.

```
page_content='54제2장 미래상과 목표
5. 기술발전에 선제적 대응, '미래교통 인프라 구축'1) 배경미래교통수단 도입에 따른 도시 활동 변화 및 공간구조 재편 대비ŸŸ빠르게 진화하는 미래 신 교통수단은 도시공간에 새로운 변화를 일으킬 것이며, 도시 내 이동패턴의 변화를 가져올 것으로 예측된다. 이러한 점을 고려하여 도시 내 다양한 통행행태가 공존하는 포용적인 교통체계 구축을 준비해야 한다.
(... 생략 ...)
```

3. 가져온 청크 중 '서울시 자율주행 계획'과 관련된 문서만 찾아서 filtered_docs에 담습니다. 각 청크는 retrieval_grader로 연관성을 판단하여 결과를 받아 is_relevant에 담습니다. 결과가 yes인 경우에만 해당 청크가 filtered_docs 리스트에 추가됩니다.

관련된 청크만 리스트에 추가하기 rag_with_langgraph.ipynb (11)

```
filtered_docs = []

for i, doc in enumerate(documents):
    print(f"Document {i + 1}:")
    is_relevant = retrieval_grader.invoke({"question": question, "document": doc.page_
content})
    print(is_relevant)
    print(doc.page_content[:200])
    print("==================================\n\n")

    if is_relevant.binary_score == "yes":
        filtered_docs.append(doc)

print(f"Filtered documents: {len(filtered_docs)}")
```

이 셀을 실행해 보니 다음과 같은 결과가 나옵니다. 서울시 자율주행 관련 내용을 5개 검색했지만, 관련 있는 내용은 3개뿐이었습니다. 언어 모델이 판단한 것이므로 반드시 3개가 아닐 수도 있어 리트리버로 가져온 문서를 살펴보니 판단을 잘 내렸습니다.

```
Document 1:
binary_score='yes'
54제2장 미래상과 목표
(... 생략 ...)
Document 3:
binary_score='no'
제1절 도시공간구조105기능이 공존하는 미래 도심으로 조성-국제 디지털 금융중심지인 여의도·영등포는 한강을 중심으로 한 글로벌 혁신코어 조성-국제 업무 중심지인 강남은 영동대로와 경부간선도로의 입체복합화로 가용지를 확보하여 업무복합기능을 강화 Ÿ수도 서울의 위상 강화를 위해 "광화문~용산~영등포"축을 국가중심공간으로 조성한다.-북악산~청와대~광화문광장~서울
==================================
(... 생략 ...)
Filtered documents: 3
```

Do it! 실습 RAG 답변 생성하기

결과 파일: sec01/rag_with_langgraph.ipynb

GPT로 질문 내용과 관련된 청크들만 필터링하여 남겼으니 이제 답변을 생성할 차례입니다.

앞서 사용한 방식과 동일하게 PromptTemplate을 이용해 사용자의 질문(question)과 관련 청크(context)를 제공했을 때 답변하는 프롬프트 rag_prompt를 만들겠습니다. 같은 PromptTemplate을 사용했지만 이번에는 선언하는 방식이 달라졌습니다. 앞에서는 PromptTemplate에서 from_template()을 만들고 프롬프트 텍스트와 빈칸을 써놓았지만 이번에는 input_variables를 리스트로 따로 써두고 template에 rag_generate_system으로 만들어 놓은 문자열을 받도록 합니다.

✦ 둘 중 어떤 방식을 사용해도 상관없지만 다양한 방식이 있음을 여러분에게 소개하고 싶었습니다. 다른 사람이 만든 코드나 문서를 읽어야 할 수도 있으니까요.

사용자의 질문은 '서울시 자율주행 관련 계획'이고 언어 모델에게 전달할 context는 filtered_docs입니다. 이 필터링된 리스트에는 3개의 청크만 포함되어 있습니다.

RAG를 위한 프롬프트 생성하기 — rag_with_langgraph.ipynb (11)

```
### Generate
# PromptTemplate을 사용해 RAG를 위한 프롬프트 생성

rag_generate_system = """          filtered_docs
너는 사용자의 질문에 대해 주어진 context에 기반하여 답변하는 도시 계획 전문가이다.
주어진 context는 vectorstore에서 검색된 결과이다.
주어진 context를 기반으로 사용자의 question에 대해 답변하라.

==================================
question: {question}  질문
context: {context} f_docs
"""

# PromptTemplate을 생성해 question과 context를 포매팅
rag_prompt = PromptTemplate(
    input_variables=["question", "context"],
    template=rag_generate_system
)

# rag chain
rag_chain = rag_prompt | model

# 사용자 질문과 검색된 문서를 입력으로 사용해 RAG를 실행
```

```
question = "서울시 자율주행 관련 계획"

rag_chain.invoke({"question": question, "context": filtered_docs})
```

이 셀을 실행하면 다음과 같이 AIMessage로 생성된 답변을 볼 수 있습니다.

```
AIMessage(content='서울시는 자율주행 교통수단 도입을 위해 단계적으로 계획을 마련하고 있습니다. 2030년까지는 간선도로급 이상의 도로에서 자율주행 자동차가 운영될 수 있는 인프라 환경을 조성하며, 2040년까지는 서울 전역에 자율주행 차량의 운행 환경을 구축할 목표를 세우고 있습니다. 이를 통해 자율주행 차량의 수송 분담률을 10%로 달성하고자 합니다.\n\n또한 도심 항공 교통(UAM)의 기반을 다지기 위해 김포공항까지의 시범 노선 운영을 포함하여, 한강과 같은 주요 수변 공간 중심으로 광역 노선도 확보할 계획입니다. 이와 함께, 서울 전역에는 모빌리티 허브를 구축하여 다양한 교통수단과 공공서비스, 물류, 상업적 기능을 통합적으로 제공하는 거점 시설도 도입할 예정입니다.\n\n이러한 여러 계획은 새로운 도시 공공교통 체계에 대한 포괄적 접근을 통해 안전하고 효율적인 이동 환경을 만들고, 급변하는 교통 기술에 유연하게 대응하기 위한 것입니다.', additional_kwargs={'refusal': None},
(... 생략 ...)
```

13-4 그래프 정의하기

이제 RAG 에이전트에 필요한 기능을 다 만들었으니 이 기능들을 엮어서 하나의 그래프로 만들어 보겠습니다.

Do it! 실습 그래프 상태 선언하고 노드 정의하기

결과 파일: sec01/rag_with_langgraph.ipynb

앞에서 사용한 rag_with_langgraph.ipynb 파일에 이어서 작성합니다.

1. 앞서 배웠듯이 `TypedDict`를 이용해 랭그래프에서 사용할 상태를 선언합니다. `GraphState`에 필요한 정보를 담아 놓고 사용합니다. 사용자 질문을 담아 두는 `question`, 언어 모델 답변 생성 결과를 담는 `generation`, 리트리버에서 검색된 문서(청크)를 담아 놓는 `documents`로 구성합니다. 이 중 `documents`는 리트리버에서 문서를 가져온 후 관련성을 판단하여 최종 필터링한 결과를 담게 됩니다.

그래프 상태 선언하기 — rag_with_langgraph.ipynb (13)

```
from typing import List
from typing_extensions import TypedDict

class GraphState(TypedDict):
    question: str         # 사용자 질문
    generation: str       # LLM 생성 결과
    documents: List[str]  # 검색된 문서
```

앞서 테스트한 기능을 랭그래프의 노드로 구현하기 위해 함수로 만들어 보겠습니다.

2. `route_question` 함수를 만듭니다. 이 함수는 `state` 매개변수를 받고 질문 내용을 바탕으로 RAG가 필요하면 `vector_store`를, 일반 대화 내용이면 `casual_talk`를 반환합니다.

route_question 함수 만들기 rag_with_langgraph.ipynb (14)

```
def route_question(state):      Node
    """
    사용자 질문을 vectorstore 또는 casual_talk로 라우팅합니다.

    Args:
        state (dict): 현재 graph state

    return:
        state (dict): 라우팅된 데이터 소스와 사용자 질문을 포함하는 새로운 graph state
    """
    print('------ROUTE------')
    question = state['question']
    route = question_router.invoke({"question": question})


    print(f"---Routing to {route.datasource}---")
    return route.datasource
```

3. 다음은 retrieve 노드를 정의합니다. 이 노드는 리트리버에서 state의 question으로 검색을 합니다. 그 검색된 결과를 딕셔너리 형태로 반환하여 graph state를 업데이트합니다.

retrieve 노드 정의하기 rag_with_langgraph.ipynb (15)

```
def retrieve(state):
    """
    vectorstore에서 질문에 대한 문서를 검색합니다.

    Args:
        state (dict): 현재 graph state

    return:
        state (dict): 검색된 문서와 사용자 질문을 포함하는 새로운 graph state
    """
    print('------RETRIEVE------')
    question = state['question']

    # Retrieve documents
    documents = retriever.invoke(question)
    return {"documents": documents, "question": question}
```

4. retrieve 노드에서 업데이트된 state의 documents 중에서 question과 연관성이 있는 문서만 필터링하여 filtered_docs에 담고 state의 documents를 filtered_docs로 업데이트하는 grade_documents 노드를 구현합니다.

grade_documents 노드 정의하기 — rag_with_langgraph.ipynb (16)

```
def grade_documents(state):
    """
    검색된 문서를 평가하여 질문과 관련성이 있는지 확인합니다.

    Args:
        state (dict): 현재 graph state

    return:
        state (dict): 관련성이 있는 문서와 사용자 질문을 포함하는 새로운 graph state
    """
    print('------GRADE------')
    question = state['question']
    documents = state['documents']
    filtered_docs = []

    for i, doc in enumerate(documents):
        is_relevant = retrieval_grader.invoke({"question": question, "document": doc.
page_content})
        if is_relevant.binary_score == "yes":
            filtered_docs.append(doc)
    return {"documents": filtered_docs, "question": question}
```

5. 다음으로 generate 노드를 정의합니다. 이 노드는 앞 단계의 grade_documents 노드에서 필터링한 state의 documents를 이용해 최종 답변을 생성하는 역할입니다.

generate 노드 정의하기 — rag_with_langgraph.ipynb (17)

```
def generate(state):
    """
    LLM을 사용하여 문서와 사용자 질문에 대한 답변을 생성합니다.

    Args:
        state (dict): 현재 graph state

    return:
        state (dict): LLM 생성 결과와 사용자 질문을 포함하는 새로운 graph state
```

```
    """
    print('------GENERATE------')
    question = state['question']
    documents = state['documents']
    generation = rag_chain.invoke({"question": question, "context": documents})
    return {
        "documents": documents,
        "question": question,
        "generation": generation
    }
```

6. RAG가 필요 없는 일상적인 질문에 답하는 `casual_talk` 노드를 만듭니다.

casual_talk 노드 정의하기 — rag_with_langgraph.ipynb (18)

```
def casual_talk(state):
    """
    일상 대화를 위한 답변을 생성합니다.

    Args:
        state (dict): 현재 graph state

    return:
        state (dict): 일상 대화 결과와 사용자 질문을 포함하는 새로운 graph state
    """
    print('------CASUAL TALK------')
    question = state['question']
    generation = model.invoke(question)
    return {
        "question": question,
        "generation": generation
    }
```

Do it! 실습 StateGraph 만들기

결과 파일: sec01/rag_with_langgraph.ipynb

랭그래프를 사용하려면 StateGraph를 만들고 노드들을 등록한 후, 각 노드들의 연결 관계를 정의해야 합니다. 앞에서 만든 노드들을 등록할 StateGraph를 만들어 보겠습니다.

1. 앞에서 question, generation, documents를 담아 둘 수 있도록 했던 GraphState를 이용해 StateGraph를 만듭니다. 이 객체의 이름은 workflow로 했습니다.

StateGraph 객체 생성하기 — rag_with_langgraph.ipynb (19)

```
from langgraph.graph import START, StateGraph, END

workflow = StateGraph(GraphState) # StateGraph 객체 생성
```

2. 먼저 만든 함수들을 .add_node를 사용해 workflow의 노드로 등록합니다.

노드 등록하고 연결하기 — rag_with_langgraph.ipynb (20)

```
# 노드를 정의
workflow.add_node("retrieve", retrieve)
workflow.add_node("grade_documents", grade_documents)
workflow.add_node("generate", generate)
workflow.add_node("casual_talk", casual_talk)
```

3. 그 노드들을 .add_edge로 연결합니다. 그리고 조건에 따라 경로를 선택하는 기능을 제공하는 .add_conditional_edges 메서드를 사용합니다. 이전에 만든 route_question은 질문 내용에 답변하는데 RAG가 필요하면 vectorstore를, 일반 대화 내용이면 casual_talk를 반환하도록 되어 있습니다. 그 결과가 어떻게 되는지에 따라 그래프의 경로가 결정됩니다. 만약 반환값이 vectorstore면 retrieve 노드로, casual_talk면 casual_talk 노드로 진행합니다. casual_talk 노드로 진행되면 답변을 생성하고 바로 END 노드를 만나고, retrieve 노드로 진행되면 retrieve → grade_documents → generate → END 경로로 연결됩니다. 이 그래프를 app이라는 이름으로 컴파일합니다.

그래프 정의하기 — rag_with_langgraph.ipynb (21)

```
workflow.add_conditional_edges(
    START,
    route_question,
    {
        "vectorstore": "retrieve",
        "casual_talk": "casual_talk"
```

```
    }
)
workflow.add_edge("casual_talk", END)
workflow.add_edge("retrieve", "grade_documents")
workflow.add_edge("grade_documents", "generate")
workflow.add_edge("generate", END)

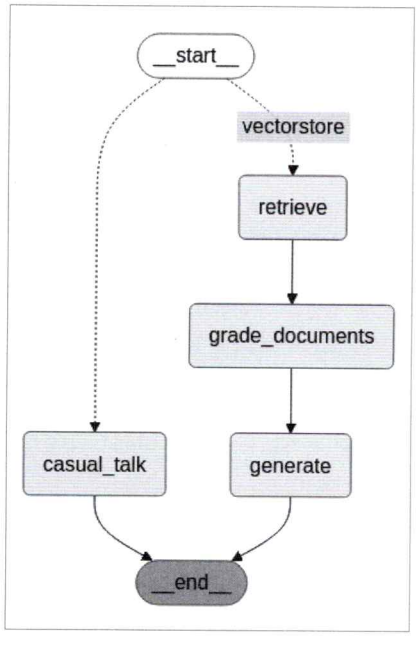
app = workflow.compile()  # workflow를 컴파일
```

4. 컴파일된 결과를 그래프로 그려 보면 다음과 같습니다.

그래프 출력하기 rag_with_langgraph.ipynb (22)

```
from IPython.display import Image, display

try:
    display(Image(app.get_graph().draw_mermaid_png()))
except Exception:
    #  실패 시 통과
    pass
```



Do it! 실습 멀티에이전트 테스트하기

결과 파일: sec01/rag_with_langgraph.ipynb

이제 필요한 경우에만 RAG로 답변을 생성하고, 그렇지 않은 일상적인 질문에는 RAG 없이 답변을 생성하는지 확인해 봅시다.

1. 서울시 자율주행 계획에 대해 app 워크플로에 물어보겠습니다.

멀티에이전트에게 질문하기 rag_with_langgraph.ipynb (23)

```
inputs = {
    "question": "서울시 자율주행 계획"
}

app.invoke(inputs)  # workflow를 실행
```

이 셀을 실행해 보면 다음처럼 잘 답변해 줍니다. 최종 답변 결과와 사용한 청크 정보를 모두 볼 수 있습니다.

```
------ROUTE------
---Routing to vectorstore---
------RETRIEVE------
------GRADE------
------GENERATE------

{'question': '서울시 자율주행 계획',
 'generation': AIMessage(content='서울시는 자율주행 자동차를 포함한 미래교통수단의 도입을 위해 2030년까지 간선도로급 이상 도로에서 자율주행 자동차가 운영될 수 있는 도로 인프라 환경을 조성할 계획입니다. 더 나아가, 2040년까지 서울 전역에서 자율주행차량의 운행 환경을 구축하고 수송 분담률 10%를 달성하는 것을 목표로 하고 있습니다. \n\n이를 위해 서울시는 도시 인프라 확충에 대한 지원이 필요하며, 자율주행차량과 같은 새로운 교통 수단의 정착을 위한 전략적 계획이 요구됩니다. 또한, 도심 항공교통(UAM) 기반을 마련하고, 모빌리티 허브를 서울 전역에 구축하여 다양한 교통수단과 서비스의 통합을 도모할 예정입니다. \n\n이러한 노력은 서울시의 교통 체계가 기존 교통수단과의 조화를 이루고, 안전하고 효율적인 통행을 가능하게 하는 데 기여할 것으로 기대됩니다.', additional_kwargs ... 생략 ...
 'documents': [Document(id='74137436-65a1-4316-94cd-0ff38c7744f1', metadata={'author': 'SI', 'creationdate': '2024-12-12T18:16:11+09:00', 'creator': 'Hwp 2020 11.0.0.5178', 'moddate': '2024-12-12T18:16:11+09:00', 'page': 61, 'page_label': '62', 'pdfversion': '1.4', 'producer': 'Hancom PDF 1.3.0.542', 'source': '../data\\2040_seoul_plan.pdf', 'total_pages': 205}, page_content='54제2장 미래상과 목표\n5. 기술발전에 선제적 대응, '미래교통 인프라 구축'1) 배경미래교통수단 도입에 따른 도시 활동 변화 및 공간구조 재편 대비Ÿ빠르게 진화하는 미래 신 교통수단은 ... 생략 ...
```

2. 일상적인 내용을 물어보면 RAG 쪽 워크플로를 따르지 않는지 확인해 봅시다.

워크플로 확인하기 rag_with_langgraph.ipynb (24)

```
inputs = {
    "question": "잘 지내고 있어?"
}

app.invoke(inputs)  # workflow를 실행
```

이 셀을 실행해 보면 잘 지내고 있다고 간단하게 답하네요.

```
------ROUTE------
---Routing to casual_talk---
------CASUAL TALK------
{'question': '잘 지내고 있어?',
 'generation': AIMessage(content='네, 잘 지내고 있습니다! 당신은 어떻게 지내고 계신가요?',
additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_
tokens': 19, 'prompt_tokens': 13, 'total_tokens': 32, 'completion_tokens_details':
{'audio_tokens': 0, 'reasoning_tokens': 0, 'accepted_prediction_tokens': 0, 'rejected_
prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens'
: 0}}, 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint': 'fp_0705bf87c0',
'finish_reason': 'stop', 'logprobs': None}, id='run-addafec5-210c-4892-9360-
051f39098de6-0', usage_metadata={'input_tokens': 13, 'output_tokens': 19, 'total_tokens':
32, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details':
{'audio': 0, 'reasoning': 0}})}
```

3. 스트림 방식으로 출력하고 싶다면 다음과 같이 작성합니다.

스트림 방식으로 출력하기 rag_with_langgraph.ipynb (25)

```
inputs = {
    "question": "서울시의 자율주행 차량 계획은 무엇이 있나요?"
}

for msg, meta in app.stream(inputs, stream_mode='messages'):
    print(msg.content, end='')
```

이 셀을 실행하면 결과가 스트림 방식으로 출력됩니다.

```
------ROUTE------
{"datasource":"vectorstore"}---Routing to vectorstore---
------RETRIEVE------
------GRADE------
{"binary_score":"yes"}{"binary_score":"yes"}{"binary_score":"yes"}{"binary_score":"no"}
{"binary_score":"no"}------GENERATE------
서울시의 자율주행 차량 관련 계획은 다음과 같습니다.

1. **운영 체계 구축**: 서울시는 자율주행 자동차의 본격 운영을 위한 체계 구축을 계획하고 있습니다. 단계적으로 2030년까지 간선도로급 이상 도로에서 자율주행차가 운영될 수 있는 도로 인프라 환경을 조성할 예정이며, 2040년까지는 서울 전역에서 자율주행 차량의 운행 환경을 구축할 목표를 가지고 있습니다.

2. **수송 분담률 목표**: 2040년까지 자율주행차량의 수송 분담률을 10% 달성하는 것을 목표로 하고 있습니다.

3. **도심 항공 교통(UAM)**: 서울은 도심 항공교통 시스템을 마련하고, 김포공항과의 연결을 포함한 시범 노선을 운영하며, 장기적으로 광역 노선도 확장할 계획입니다.

4. **모빌리티 허브 구축**: 서울 전역에 모빌리티 허브를 구축하여, 기존의 교통 시스템과 새로운 교통수단을 연계하고 다양한 공공 서비스와 상업 기능이 결합된 지역 거점을 조성할 예정입니다.

이러한 계획들은 기술 발전에 선제적으로 대응하고, 미래의 도시 활동 변화에 따라 보다 포용적인 교통체계를 마련하기 위한 노력의 일환으로 진행되고 있습니다.
```

이 장에서는 랭그래프를 활용해 상황에 따라 다른 AI 에이전트를 사용하는 방법을 배웠습니다. 여기에 앞서 배웠던 웹 검색 에이전트를 추가로 붙일 수도 있고, RAG의 답변을 확인하고 적절한 답변을 작성하지 못했을 때 다른 검색어로 검색한 뒤 글을 작성하는 프로그램을 만들 수도 있습니다. 상상력을 발휘해 여러분만의 AI 에이전트를 구현해 보세요!